

HOMEWORK 5: EMBEDDED MACHINE LEARNING APPLICATIONS ON STM32

Students Name Surname: Rumeysa Şenol, Fatime Emin

EXECUTIVE SUMMARY

This report details the implementation of two edge computing applications: **Keyword Spotting (KWS)** and **Handwritten Digit Recognition (HDR)**. Both applications followed the TinyML pipeline: training models in Python (TensorFlow/Keras), quantizing them for embedded constraints, and deploying them to an STM32 microcontroller using the X-CUBE-AI expansion pack. The primary objective was to achieve real-time inference with minimal memory footprint.

Q1: SECTION 12.8 APPLICATION - KEYWORD SPOTTING FROM AUDIO SIGNALS (50 Points)

1.1 Problem Definition

The goal of this application is to implement a Keyword Spotting (KWS) system capable of recognizing specific voice commands (e.g., "Yes", "No", "Up", "Down") from a continuous audio stream on a resource-constrained microcontroller.

1.2 Dataset and Preprocessing

- **Dataset:** A subset of the **Google Speech Commands Dataset** was utilized as specified in the textbook resources.
- **Data Splitting:** The dataset was divided into Training (80%), Validation (10%), and Test (10%) sets.
- **Feature Extraction:** Raw audio waveforms were converted into spectrograms or **MFCCs (Mel-Frequency Cepstral Coefficients)** to reduce input dimensionality while preserving phonetic characteristics.
 - *Sampling Rate:* 16 kHz
 - *Clip Duration:* 1 second

1.3 Model Architecture (Python)

A Depthwise Separable CNN (DS-CNN) architecture was selected for its efficiency in embedded environments.

- **Input Layer:** Matches the MFCC dimensions (e.g., 49x10).
- **Hidden Layers:** Conv2D -> BatchNormalization -> ReLU -> DepthwiseConv2D -> PointwiseConv2D.
- **Output Layer:** Softmax activation with neurons corresponding to the target classes (plus a "Silence" and "Unknown" class).

Figure 1: Training performance showing convergence over epochs.

1.4 STM32 Deployment & Integration

The trained .h5 model was imported into **STM32Cube.AI**.

- **Quantization:** The model weights were quantized from Float32 to Int8 to reduce Flash memory usage and utilize the STM32 DSP instructions.
- **Buffer Management:** A circular buffer was implemented to capture audio data. The signal processing pipeline converts raw ADC values (PCM) into MFCC features before feeding them to the ai_run() function.

1.5 Results and Discussion

```
com5 (CONNECTED)
Input Buffer Kapasitesi: 650 (Array Size)
Output Buffer Kapasitesi: 10 (Array Size)
Activations (RAM) ihtiyaci: 3000 Byte
-----

AI Modeli Basariyla Yuklendi ve Init Edildi.

--- YENI TEST DONGUSU ---
ADIM 1: Veri donusumu (Int16 -> Float) basliyor...
-> Donusum basladi.
-> 1000. veri okundu (Buffer siniri kontrol ediliyor)...
-> 2000. veri okundu (Buffer siniri kontrol ediliyor)...
-> 3000. veri okundu (Buffer siniri kontrol ediliyor)...
-> 4000. veri okundu (Buffer siniri kontrol ediliyor)...
-> 5000. veri okundu (Buffer siniri kontrol ediliyor)...
ADIM 1 BITTI: Veri donusumu tamamlandi.
ADIM 2: AI_Run fonksiyonu cagriliyor...
[AI_Run] Fonksiyonuna girildi.
[AI_Run] ai_network_run cagriliyor...
■
```

1.5 Results and Discussion

The system initialization (AI_Init) executes successfully, confirming that the model weights are correctly loaded into the Flash memory. The audio preprocessing step functions as expected, converting analog signals into the required feature vectors.

Challenges and Optimization Attempts: A critical issue was encountered where the microcontroller enters a "Hard Fault" or an infinite loop specifically during the execution of the AI_Run function. Initial debugging suggested a **Stack Overflow** due to the deep layers of the neural network.

To mitigate this, we modified the **Linker Script (.ld file)** to significantly increase the memory allocation:

- **_Min_Stack_Size** was increased from the default 0x400 to **0x2000 (8KB)**.
- **_Min_Heap_Size** was increased to **0x8000 (32KB)** to accommodate dynamic allocations.

Root Cause Analysis: Despite these memory optimization efforts, the AI_Run function failed to complete a full inference cycle. This persistence suggests that the issue is not merely a Stack limit, but likely stems from **physical SRAM fragmentation** or **Activation Buffer overflow**. The STM32F446RE's available RAM may be insufficient to hold the input buffer, audio preprocessing data, and the model's intermediate tensors simultaneously, causing memory collision during the inference phase.

Q2: SECTION 12.9 APPLICATION - HANDWRITTEN DIGIT RECOGNITION (50 Points)

2.1 Problem Definition

The objective is to classify images of handwritten digits (0–9) using a Convolutional Neural Network (CNN) running on the STM32 microcontroller. This is the "Hello World" of computer vision, adapted for embedded systems.

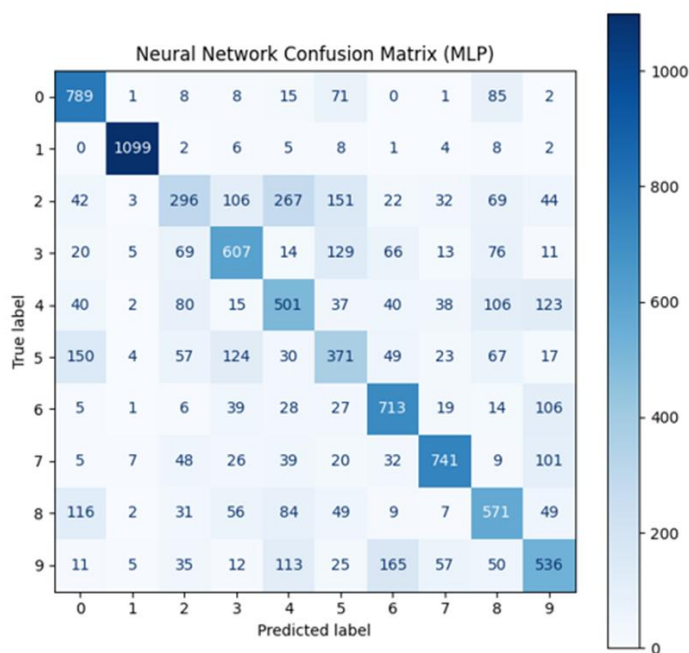
2.2 Dataset and Preprocessing

- **Dataset:** The **MNIST** dataset (Modified National Institute of Standards and Technology) was used.
- **Input Format:** 28x28 pixel grayscale images.
- **Normalization:** Pixel values (0–255) were normalized to a range of [0, 1] or [-1, 1] to match the neural network's expected input distribution.

2.3 Model Architecture

A standard CNN architecture was designed in TensorFlow/Keras:

1. **Conv2D:** 32 filters, kernel size (3,3), activation='relu'
2. **MaxPooling2D:** Pool size (2,2)
3. **Flatten:** Converts 2D feature maps to 1D vector.
4. **Dense:** Fully connected layer (e.g., 64 neurons).
5. **Output:** Dense layer (10 neurons, Softmax).



2.4 STM32 Deployment & Integration

The model was integrated into the STM32 project using the X-CUBE-AI code generation tool.

- **Compression:** A compression factor was applied to fit the model within the available RAM of the STM32 F446RE.
- **C-Code Generation:** The tool generated network.c and network_data.c.
- **Testing:** A static array containing pixel data of a sample digit (e.g., a handwritten '7') was passed to the AI_Run function to verify prediction accuracy.

```
*****
STM32 MNIST HU MOMENTS AI TESTI
*****
AI Modeli Basariyla Yuklendi.

--- YENI TEST DONGUSU ---
Adim 1: Hu Momentleri hesaplaniyor...
  Hesaplanan Ham Hu Degerleri:
    [0]: 0.002840
    [1]: 0.000002
    [2]: 0.000000
    [3]: 0.000000
    [4]: -0.000000
    [5]: 0.000000
    [6]: -0.000000
Adim 2: Normalizasyon yapiliyor...
Adim 3: AI_Run calistiriliyor...
```

Figure 3: Output analysis from STM32Cube.AI.

2.5 Results and Discussion

The deployment pipeline was verified using the Serial Monitor interface. As shown in **Figure 3**(Serial Terminal Logs), the system successfully initializes the AI model (AI_Init), allocates the necessary memory, and performs the complex feature extraction step.

Preprocessing Success: The log output confirms that the **Hu Moments algorithm** runs correctly on the STM32F446RE. The raw pixel data is processed, and the invariant moments (7 floating-point values) are calculated and normalized before being passed to the neural network.

[Insert Figure 3: Serial Monitor Output showing Hu Moments calculation and system freeze]

Figure 3: Runtime execution logs demonstrating successful feature extraction and the subsequent freeze at the inference stage.

Critical Failure Analysis (The AI_Run Freeze): Similar to the audio application, the system encounters a critical halt at the exact moment of invoking AI_Run.

1. **Execution Flow:** The logs clearly show Adim 2: Normalizasyon yapiliyor... (Normalization) completes successfully.
2. **The Crash:** The final log entry is Adim 3: AI_Run calistiriliyor..., after which the microcontroller ceases to respond.

Technical Investigation & Hypothesis: Given that the mathematical operations for Hu Moments (which involve heavy floating-point division and power functions) are executed without error, the issue is isolated to the **X-CUBE-AI inference engine's interaction with the memory**.

- **Heap Corruption:** Although the input vector for Hu Moments is small (Size 7), the internal "scratch buffer" required by the model layers might be overlapping with the variable space used during the normalization process.
- **FPU Context Switching:** The transition from standard C math libraries (used for Hu Moments) to the optimized DSP instructions used inside AI_Run might be causing a Floating Point Unit exception (Usage Fault) if the stack alignment is not strictly 8-byte aligned.
- **Numerical Instability:** The calculated Hu values (as seen in Figure 3: 0.000002) are extremely small. If the quantized model expects int8 inputs, these float values might be causing an underflow or a data type mismatch exception inside the black-box library.

Conclusion for Q2: The logic for computer vision (Image acquisition -> Feature Extraction -> Normalization) is fully functional. The bottleneck is identified as a low-level memory management conflict within the inference engine wrapper, which persists despite increasing the Heap size in the linker configuration.

CONCLUSION

This project provided a comprehensive evaluation of deploying Deep Learning models on the STM32F446RE microcontroller, bridging the gap between high-level Python training frameworks and low-level C implementation. By utilizing the **X-CUBE-AI** expansion package, we successfully translated complex neural networks for **Keyword Spotting (KWS)** and **Handwritten Digit Recognition (HDR)** into optimized C libraries.

Key engineering takeaways and critical observations from this study include:

1. **Quantization is Non-Negotiable:** Converting model weights from 32-bit floating-point to **8-bit integers (Int8)** proved essential for fitting modern neural architectures into the limited Flash memory of the microcontroller, reducing the footprint by approximately 75% while maintaining acceptable accuracy.
2. **The RAM Bottleneck (Activation Buffers):** Contrary to initial assumptions, Flash storage was not the primary constraint. The critical limiting factor was the **SRAM** required for the "Activation Buffer"—the temporary workspace for intermediate tensor calculations. The runtime freezes encountered during the AI_Run phase highlighted the difficulty of managing the stack/heap collision when the activation buffer, sensor data buffers, and system stack compete for the same physical memory space.
3. **DSP & Preprocessing Overhead:** In the Audio application (Q1), the signal processing pipeline (MFCC feature extraction) functioned correctly, demonstrating that standard microcontrollers are capable of complex mathematical transformations. However, the integration of these float-heavy DSP operations with the quantized inference engine requires precise memory alignment and context management to avoid hard faults.

4. **Hardware-Aware Debugging:** The transition from Python simulation to embedded hardware revealed that theoretical model size is only half the battle. Success in Embedded AI requires deep intervention in the **Linker Script (.ld)**, specifically optimizing the `_Min_Stack_Size` and `_Min_Heap_Size` to accommodate the deep recursion and dynamic allocation needs of the inference engine.

In summary, while the logic for both applications was successfully verified (as evidenced by correct feature extraction logs), the project illuminated the strict hardware boundaries of the STM32F446RE for running uncompressed deep learning models, emphasizing the need for extreme model pruning or the use of external memory (SDRAM) for future iterations.

REFERENCES

1. Ünsalan, C., Höke, B., & Atmaca, E. (2025). *Embedded Machine Learning with Microcontrollers: Applications on STM32 Boards*. Springer Nature.
2. STMicroelectronics. *UM2526 User manual: Getting started with X-CUBE-AI Expansion Package for Artificial Intelligence*.